

Biomedical Knowledge Graph Triplet Extraction from PubMed Sentences

Abstract

This project focused on extracting biomedical knowledge graph triplets from annotated PubMed-style sentences. Each triplet contains a subject entity, a relation type, and an object entity, which can help turn unstructured biomedical writing into a more structured knowledge graph format. I worked with two datasets: ADKG, which focuses on Alzheimer’s disease and related neurodegenerative disorders, and MDKG, which focuses on mental disorders.

I first performed exploratory data analysis to understand the structure of the datasets, including entity types, relation types, sentence density, and relation distance. Then I converted the original character-offset annotations into a format that could be used by SpERT, a span-based entity and relation extraction model. For the main baseline, I trained separate SpERT models on ADKG and MDKG using a BERT-small encoder. On the test sets, the SpERT baseline reached a relation F1 score of 28.86 on ADKG and 24.95 on MDKG when named entity classification was required.

For the extension, I compared this discriminative approach with a generative triplet extraction setup using T5-small. I converted the data into an instruction-based text-to-text format, where the model generated triplets directly as text. The generative model reached a strict triplet F1 of 19.79 and a relaxed triplet F1 of 26.60 on a combined test subset. Overall, the results show that biomedical triplet extraction is possible, but relation extraction remains challenging, especially in dense sentences with multiple entities and longer-distance relations.

1. Introduction

Biomedical research articles contain a large amount of useful information, but most of that information is written as unstructured text. This makes it harder to directly use the information in downstream systems, such as search tools, clinical decision support systems, or biomedical knowledge graphs. One way to make this information more structured is to extract triplets from text. In this project, each triplet has a subject entity, a relation type, and an object entity. For example, the phrase “Alzheimer’s disease (AD)” can produce the triplet `AD | abbreviation_for | Alzheimer's disease`.

The main goal of this project was to build and evaluate a pipeline for extracting these kinds of biomedical triplets from PubMed-style sentences. I worked with two annotated datasets. The first dataset, ADKG, focuses on Alzheimer’s disease and related neurodegenerative disorders. The second dataset, MDKG, focuses on mental disorders, including depression, schizophrenia, bipolar disorder, PTSD, OCD, and related topics. Both datasets include entity annotations and relation annotations, which made it possible to train supervised models for named entity

recognition and relation extraction.

I approached the project in two main stages. First, I built a discriminative extraction pipeline using SpERT. SpERT predicts entity spans, entity types, and relations between entities. This served as the main baseline because it is designed for joint entity and relation extraction. Before training, I had to convert the original character-level annotations into token-level spans that SpERT could use.

Second, I added a generative extension using T5-small. Instead of predicting entities and relations through a fixed classification framework, the generative model was trained to read a sentence and produce triplets as text. This allowed me to compare a structured discriminative model with a more flexible text-generation approach. Since generative outputs do not always match the gold text exactly, I evaluated the T5 model with both strict and relaxed triplet matching.

The rest of this report explains the dataset structure, the exploratory analysis, the preprocessing steps, the SpERT baseline, the T5 extension, and the main results. I also include error analysis examples to show where each approach worked well and where it struggled.

2. Dataset Description and Exploratory Analysis

The project used two annotated biomedical datasets: ADKG and MDKG. ADKG focuses on Alzheimer’s disease and related neurodegenerative disorders, while MDKG focuses on mental disorders. Both datasets are based on PubMed-style biomedical sentences and include entity annotations and relation annotations. Each example contains a sentence, a list of entities, and a list of directed relations between those entities.

Before training any models, I spent time checking the structure of the data and summarizing the main patterns. This step was important because relation extraction depends heavily on the density of the sentence, the number of possible entity pairs, and the types of relations that appear most often.

2.1 Dataset Split Summary

Dataset	Split	Sentences	Entities	Relations	Avg Enti- ties/Sentences	Avg Rela- tions/Sentence
ADKG	train	5605	14611	3939	2.61	0.70
ADKG	dev	1206	3099	788	2.57	0.65
ADKG	test	1220	3149	769	2.58	0.63
MDKG	train	4825	20737	7559	4.30	1.57
MDKG	dev	941	3963	1570	4.21	1.67
MDKG	test	912	3960	1431	4.34	1.57

The first clear difference between the datasets was that MDKG was much denser than ADKG. Even though MDKG had fewer sentences overall, it had more entities and relations per sentence. For example, the MDKG training set had an average of 4.30 entities and 1.57 relations per sentence, while the ADKG training set had an average of 2.61 entities and 0.70 relations per sentence. This suggested that MDKG might be harder in a different way: each sentence usually had more possible entity pairs for the model to consider.

2.2 Entity Type Distribution

ADKG was mostly centered on a smaller group of entity types. The most common entity type was `disease`, followed by `gene`, `drug`, `method`, `other`, and `mutation`.

ADKG Entity Type	Count
disease	8579
gene	4920
drug	2857
method	2736
other	1540
mutation	227

MDKG contained a wider range of entity types. It still had many `disease` and `method` entities, but it also included categories such as `Health_factors`, `physiology`, `region`, `signs`, and `symptom`.

MDKG Entity Type	Count
disease	8951
method	7972
Health_factors	4498
drug	2145
physiology	1615
gene	1298
region	1161
signs	557
symptom	463

This difference matters because the model has to learn not only where entities are, but also what type each entity belongs to. A dataset with more entity categories can make named entity recognition harder, especially when some categories appear much less often than others.

2.3 Relation Type Distribution

The two datasets also had different relation distributions. In ADKG, the most common relation was `abbreviation_for`, followed by `associated_with`, `hyponym_of`, and `treatment_for`.

ADKG Relation Type	Count
<code>abbreviation_for</code>	1870
<code>associated_with</code>	1177
<code>hyponym_of</code>	649
<code>treatment_for</code>	592
<code>risk_factor_of</code>	458
<code>help_diagnose</code>	355
<code>characteristic_of</code>	346
<code>treatment_target_for</code>	49

In MDKG, the most common relation was `associated_with`, followed by `treatment_for`, `occurs_in`, `risk_factor_of`, `hyponym_of`, and `abbreviation_for`.

MDKG Relation Type	Count
<code>associated_with</code>	2704
<code>treatment_for</code>	1260
<code>occurs_in</code>	1260
<code>risk_factor_of</code>	1205
<code>hyponym_of</code>	1183
<code>abbreviation_for</code>	1114
<code>characteristic_of</code>	719
<code>help_diagnose</code>	592
<code>located_in</code>	523

This helped explain some of the later model results. Abbreviation relations are often easier because they follow clearer surface patterns, such as an acronym appearing near a longer phrase. Relations like `associated_with`, `risk_factor_of`, and `treatment_for` can be harder because they depend more on the meaning of the sentence.

2.4 Sentence Density and Sparsity

I also checked how many sentences had zero entities or zero relations. This was useful because relation extraction models have to learn from both positive examples and no-relation examples.

Dataset	Split	Zero Entities (%)	Zero Relations (%)	Avg Enti- ties/Sentence	Avg Rela- tions/Sentence
ADKG	train	15.04%	64.89%	2.61	0.70
ADKG	dev	16.00%	64.76%	2.57	0.65
ADKG	test	14.26%	66.23%	2.58	0.63
MDKG	train	3.94%	46.65%	4.30	1.57
MDKG	dev	4.14%	42.93%	4.21	1.67
MDKG	test	5.37%	46.71%	4.34	1.57

ADKG was much sparser than MDKG. Around 65% of ADKG sentences had no annotated relations, while MDKG had about 43% to 47% of sentences with no relations. This means ADKG gave the model many no-relation examples, while MDKG had more sentences where at least one relation needed to be extracted.

2.5 Common Head-Tail Entity Type Pairs

Since each relation connects a head entity to a tail entity, I also counted the most common head-to-tail entity type pairs. This gave a better sense of what kinds of triplets the models needed to learn.

The most common ADKG relation entity-type pairs were:

ADKG Head Type → Tail Type	Count
disease → disease	1534
gene → gene	598
other → other	523
other → disease	431
drug → disease	424
method → disease	404
gene → disease	395

The most common MDKG relation entity-type pairs were:

MDKG Head Type → Tail Type	Count
disease → disease	1935
method → disease	1067
Health_factors → disease	861
drug → disease	700
disease → Health_factors	679
method → method	656

Both datasets had many disease-to-disease relations, but MDKG had more variety in the kinds of entity pairs that appeared often. This again suggested that MDKG may be more complex because the model has to choose from a wider range of possible relation patterns.

2.6 Relation Distance

Finally, I measured how far apart the related entities were in each sentence. I used the character gap between the head entity and tail entity as a simple distance measure. This helped connect the EDA to the project question about long-range dependencies.

Dataset	Split	Avg Distance	Median Distance	Max Distance	Long Distance (%)
ADKG	train	35.70	20.00	343	8.68%
ADKG	dev	33.40	20.00	237	7.49%
ADKG	test	35.31	18.00	292	8.58%
MDKG	train	46.44	30.00	502	12.04%
MDKG	dev	51.42	32.00	1068	13.12%
MDKG	test	49.73	31.00	613	14.26%

MDKG relations were usually farther apart than ADKG relations. For example, the MDKG test set had an average relation distance of 49.73 characters, compared with 35.31 for ADKG. MDKG also had a higher percentage of long-distance relations. This may make relation extraction harder because the model has to connect entities across longer parts of the sentence instead of relying only on nearby text.

Overall, the EDA showed that ADKG and MDKG are not just two versions of the same task. ADKG is sparser and has many no-relation sentences, while MDKG is denser, has more entity types, and has longer-distance relations. These differences helped frame the later model results.

3. Preprocessing

After checking the datasets, I converted the data into two different model-ready formats. The first format was for SpERT, which needs token-level entity spans and structured relation labels. The second format was for the generative T5 extension, which needs input-output text pairs.

3.1 Preprocessing for SpERT

The original ADKG and MDKG files used character-level entity offsets. Each entity had a start and end position based on characters in the original sentence. SpERT, however, expects entity spans at the token level. Because of that, I wrote a conversion script to map each character span to the matching token span.

The conversion process followed these steps:

1. Load the raw ADKG and MDKG JSON files.
2. Tokenize each sentence using a simple regular-expression tokenizer.
3. Keep the character start and end position for every token.
4. Match each entity’s character span to the token or tokens that overlap with it.
5. Convert each relation so that the head and tail point to entity index numbers instead of full entity dictionaries.
6. Save the converted train, dev, and test files in SpERT-style JSON format.
7. Save a shared entity and relation type file.

This step was important because a small span conversion mistake could affect both NER and relation extraction. Before training, I checked that the original entity spans matched the text and that all relation head and tail IDs pointed to valid entities. Both ADKG and MDKG passed these checks with zero span errors and zero relation ID errors.

3.2 Preprocessing for the Generative Extension

For the generative extension, I converted the same raw datasets into a text-to-text format. Instead of asking the model to classify spans and relation labels directly, I gave the model an instruction and asked it to generate triplets.

The input format was:

`Extract biomedical triplets from this sentence: [sentence text]`

The target format was:

`subject | relation | object ; subject | relation | object`

For example, a sentence containing an abbreviation relation could be converted into:

`AD | abbreviation_for | Alzheimer's disease`

If a sentence had no annotated relations, the target output was:

`none`

This format made it possible to fine-tune T5-small as a generative triplet extraction model. I created full generative train, dev, and test files for ADKG, MDKG, and the combined dataset. To keep the extension manageable, I trained the T5 model on a smaller combined subset with 3,000 training examples, 500 dev examples, and 500 test examples.

This preprocessing step also made the extension easier to compare with SpERT. SpERT treated triplet extraction as a structured prediction problem, while T5 treated it as a text generation problem.

4. SpERT Baseline Method

For the main baseline, I used SpERT as the discriminative relation extraction model. I chose SpERT because it is designed for joint entity and relation extraction. In this setup, the model first considers possible entity spans, predicts entity types for those spans, and then predicts relations between entity pairs.

This made SpERT a good fit for the core project because the datasets required both named entity recognition and relation extraction. The final goal was not just to find biomedical entities, but to connect them into structured subject-relation-object triplets.

I trained separate SpERT models for ADKG and MDKG. I first tested the pipeline with a very small BERT model to make sure the data conversion, training loop, and evaluation code all worked. After that, I used `bert-small` as the final baseline encoder. This was a practical middle ground: it was much stronger than the tiny test model, but still small enough to train quickly and reliably on Longleaf.

4.1 SpERT Training Setup

Setting	Value
Model	SpERT
Encoder	bert-small
Datasets	ADKG and MDKG trained separately
Epochs	3
Train batch size	4
Eval batch size	2
Learning rate	5e-5
Negative entity count	40
Negative relation count	40
Max span size	10
Max pairs	500

The model was evaluated using the same general categories reported by SpERT: named entity recognition, relation extraction without named entity classification, and relation extraction with named entity classification. The relation score with named entity classification is stricter because the model has to get the entity spans, entity types, and relation type correct.

5. SpERT Results

5.1 Development Set Results

Dataset	Task	Precision	Recall	F1
ADKG	NER micro	45.28	69.44	54.81
ADKG	Relations without NEC micro	34.54	40.05	37.09
ADKG	Relations with NEC micro	27.50	31.89	29.53
MDKG	NER micro	61.32	79.54	69.25
MDKG	Relations without NEC micro	32.33	30.08	31.17
MDKG	Relations with NEC micro	26.85	24.98	25.88

The development results showed that the model learned the task much better after switching from the tiny test model to **bert-small**. For both datasets, NER performance was higher than relation extraction performance. This makes sense because relation extraction is a harder task. The model must not only identify the entities, but also decide whether a relation exists between them and choose the correct relation type.

MDKG had stronger NER performance than ADKG on the development set. However, ADKG had slightly stronger relation extraction performance. This was interesting because MDKG had more entities and relations per sentence, which may have made the relation prediction step more difficult even though entity recognition itself was stronger.

5.2 Test Set Results

Dataset	Task	Precision	Recall	F1
ADKG	NER micro	47.75	68.28	56.19
ADKG	Relations without NEC micro	33.54	35.37	34.43
ADKG	Relations with NEC micro	28.11	29.65	28.86
MDKG	NER micro	59.78	77.63	67.55
MDKG	Relations without NEC micro	32.37	25.87	28.76
MDKG	Relations with NEC micro	28.08	22.45	24.95

The test results followed the same general pattern as the development results. NER was stronger than relation extraction, and relation extraction became harder when named entity classification was included. On ADKG, the model reached a relation F1 of 28.86 when entity classification was required. On MDKG, the corresponding score was 24.95.

One thing I noticed was that ADKG and MDKG had similar relation precision with named entity classification, but MDKG had lower recall. This means the MDKG model was more conservative or missed more gold relations. This fits with the earlier EDA because MDKG had denser sentences, more entity types, and longer-distance relations. In other words, even when the model found many entities, it still had to choose the correct relations from a larger and more complicated set of possible entity pairs.

Overall, the SpERT baseline gave a solid structured extraction model for both datasets. It was not perfect, but it provided a useful benchmark for the generative extension.

6. Extension: Generative Triplet Extraction with T5

For the extension, I compared the SpERT baseline with a generative triplet extraction approach. The main idea was to frame relation extraction as a text-to-text task. Instead of predicting entity spans and relation labels directly, the model received a sentence and generated the triplets as plain text.

I used T5-small for this experiment because it was small enough to train within the project timeline but still allowed me to test a different modeling style. This extension was meant to answer a simple question: how does a generative model compare with a structured discriminative model for biomedical triplet extraction?

6.1 Generative Input and Target Format

Each example was converted into an instruction-style input. The input looked like this:

`Extract biomedical triplets from this sentence: [sentence text]`

The target was a list of triplets in this format:

`subject | relation | object ; subject | relation | object`

If the sentence had no annotated relations, the target was simply:

`none`

This setup made the task easy to understand. The model had to read the sentence and either generate one or more triplets or decide that no annotated relation was present.

6.2 T5 Training Setup

Setting	Value
Model	T5-small
Training data	Combined ADKG + MDKG subset
Train examples	3000
Dev examples	500
Test examples	500
Epochs	2
Batch size	8
Learning rate	3e-4
Input max length	192
Target max length	128

I trained the T5 model on a smaller combined ADKG and MDKG subset. This kept the extension manageable while still giving the model examples from both biomedical domains. The goal was not to beat SpERT with a much larger model, but to test whether triplet extraction could be handled in a generative format.

6.3 T5 Training Loss

Epoch	Train Loss	Dev Loss
1	0.9198	0.4406
2	0.4689	0.3905

The training loss and dev loss both decreased from epoch 1 to epoch 2. This suggests that the model learned the basic output format and improved at predicting the target triplet strings. Loss alone does not prove that the generated triplets are correct, so I also evaluated the model using triplet-level precision, recall, and F1.

6.4 Generative Evaluation Method

Evaluating a generative model is different from evaluating SpERT because T5 outputs plain text. A prediction can be close to correct but still fail exact string matching because of small differences in wording, spacing, or entity span boundaries.

Because of this, I used two matching methods:

- **Strict matching:** the subject, relation type, and object had to match the gold triplet exactly after basic normalization.
- **Relaxed matching:** the relation type had to match exactly, but the subject and object only needed word overlap with the gold subject and object.

The strict score shows how often the model generated the exact gold triplet. The relaxed score gives partial credit for predictions that captured the correct relation and roughly the correct entities, even if the entity text was not exactly the same.

6.5 Generative Test Results

Model	Evaluation Data	Match Type	Precision	Recall	F1
T5-small generative	Combined test subset	Strict	24.22	16.73	19.79
T5-small generative	Combined test subset	Relaxed	32.55	22.48	26.60

The T5 model achieved a strict F1 of 19.79 and a relaxed F1 of 26.60. The relaxed score was higher, which shows that some generated triplets were close to the gold triplets but did not match exactly. The model also reached a no-relation accuracy of 77.27%, meaning it often correctly predicted **none** for sentences with no gold triplets.

The main weakness of the generative model was recall. It predicted 384 triplets while the gold set contained 556 triplets, so it missed many relations. This was especially noticeable in sentences with multiple annotated relations, where the model sometimes generated only one of the correct triplets.

7. Model Comparison

The table below compares the final SpERT test results with the generative T5 extension results.

Model	Dataset or Evalu- ation Data	Task or Match Type	Precision	Recall	F1
SpERT bert- small	ADKG test	Relations with NEC micro	28.11	29.65	28.86
SpERT bert- small	MDKG test	Relations with NEC micro	28.08	22.45	24.95
T5-small genera- tive	Combined test subset	Strict triplet match- ing	24.22	16.73	19.79
T5-small genera- tive	Combined test subset	Relaxed triplet match- ing	32.55	22.48	26.60

This comparison is not perfectly controlled because SpERT was evaluated separately on the full ADKG and MDKG test sets, while T5 was evaluated on a smaller combined test subset. Still, it gives a useful picture of the difference between the two approaches.

SpERT was stronger as a structured extraction model. This makes sense because it was designed specifically for joint entity and relation extraction. It predicts spans and relation labels in a more controlled format, so it does not have to learn the output formatting from scratch.

T5 was more flexible because it generated triplets directly as text. This made the method easier to inspect qualitatively, but it also introduced new problems. The model sometimes produced plausible relations that were not in the gold annotations, missed extra triplets in multi-relation sentences, or generated the wrong relation type. The gap between strict and relaxed F1 shows that some T5 predictions were partially correct but not exact enough for strict matching.

Overall, SpERT was the better choice for the main extraction pipeline, while T5 was a useful extension for exploring generative triplet extraction. The generative approach seems promising, but it would probably need more data, better prompting, a larger model, or stronger evaluation methods to compete with a structured extraction model.

8. Error Analysis

To better understand the results, I looked at individual prediction examples from both SpERT and T5. This was helpful because the overall F1 scores only show part of the story. The examples showed that both models could correctly extract simple relations, but they struggled in different ways.

8.1 SpERT Error Analysis

SpERT was strongest on clearer relation patterns, especially abbreviation relations. One example from ADKG involved the phrase “Montreal Cognitive Assessment (MoCA).” The gold triplet was:

`MoCA | abbreviation_for | Montreal Cognitive Assessment`

SpERT predicted the same triplet exactly. This makes sense because abbreviation relations often have a direct surface pattern where the acronym appears right next to the full term.

SpERT struggled more when the relation was expressed indirectly. In one sentence, the gold relation was:

`alpha - amylase | associated_with | AMY1A`

but SpERT predicted no relation. The sentence described a “positive correlation” between brain alpha-amylase activity and AMY1A copy number. This kind of wording may be harder because the model has to understand that the correlation expresses an association.

SpERT also had trouble with sentences containing multiple relations. For example, in one sentence about beta-secretase and Alzheimer’s disease, the gold annotations included three triplets:

`BACE1 | treatment_for | Alzheimer ' s disease`

`BACE1 | abbreviation_for | beta - secretase`

`beta - secretase | treatment_for | Alzheimer ' s disease`

SpERT correctly predicted the abbreviation relation, but missed the two **treatment_for** relations. This suggests that the model handled the simple abbreviation pattern better than the more semantic treatment relation.

Another issue was extra plausible predictions. In one example, SpERT correctly predicted:

AD | abbreviation_for | Alzheimer ' s disease

and

Alzheimer ' s disease | hyponym_of | age - associated dementia

but it also predicted:

Alzheimer ' s disease | hyponym_of | dementia

This prediction is not unreasonable, but it was not part of the gold annotation. This shows a common challenge in biomedical relation extraction: sometimes a model prediction can be logically plausible but still count as wrong because it does not exactly match the dataset annotation.

Overall, SpERT’s errors suggest that the model handled short, clear, surface-level relations better than harder relations involving indirect wording, multiple relation types, or many possible entity pairs.

8.2 T5 Error Analysis

The T5 model had a different pattern of errors because it generated triplets as text. This made the predictions easy to read, but it also introduced problems like hallucinated relations, incomplete outputs, and wrong relation types.

T5 was able to correctly generate some simple triplets. For example, for the sentence:

Alzheimer's disease (AD) belongs to one of the most multifactorial, complex and heterogeneous morbidity-leading disorders.

the gold target was:

AD | abbreviation_for | Alzheimer's disease

and T5 predicted the exact same triplet. Like SpERT, T5 seemed to handle abbreviation relations well when the acronym and full phrase were close together.

T5 also correctly predicted **none** for some no-relation examples. This mattered because many sentences, especially in ADKG, did not have annotated relations. In one example about molecular docking with chlorogenic acid, lutein, and zeaxanthin, the gold target was **none**, and T5 also predicted **none**.

However, T5 sometimes hallucinated plausible relations. In one ADKG example, the gold target was **none**, but T5 predicted:

peripheral monocytes | associated_with | trauma-induced inflammatory sequelae

This prediction sounds biologically reasonable from the sentence, but it was not part of the gold annotation. This is one weakness of generative models: they may generate something that sounds valid even when the task requires sticking exactly to the annotation scheme.

T5 also missed relations in sentences with more than one gold triplet. In one MDKG example, the gold target was:

depression | associated_with | musculoskeletal pain ; musculoskeletal pain | occurs_in | physically inactive participants

T5 predicted only:

depression | associated_with | musculoskeletal pain

This was partly correct, but incomplete. This kind of error helps explain why the T5 model had lower recall than precision.

Finally, T5 sometimes identified relevant biomedical terms but chose the wrong relation type. In one ADKG example, the gold relations connected CNS diseases with MAO and ChE using `associated_with`. T5 instead generated abbreviation relations involving MAO and ChE. This suggests that the model learned common output patterns, but it did not always understand which relation type fit the sentence.

Overall, the T5 model was useful because it showed that triplet extraction can be framed as text generation. However, it was less reliable than SpERT for exact structured extraction. Its errors were often more generative in nature: it could miss triplets, generate plausible but unannotated relations, or produce the wrong relation type.

9. Discussion

The results from this project show that biomedical triplet extraction is possible, but the task is still difficult. The clearest pattern was that named entity recognition was easier than relation extraction. For both ADKG and MDKG, the SpERT model had much higher NER F1 scores than relation F1 scores. This makes sense because relation extraction depends on multiple steps being correct at the same time. The model has to find the right entity spans, classify the entity types, decide whether a relation exists, and then choose the correct relation type.

The difference between relation extraction with and without named entity classification also shows this challenge. When entity type classification was ignored, the relation F1 scores were higher. When entity types had to be correct too, the scores dropped. This means some relation predictions had the correct spans and relation type, but failed because one or both entity types were wrong.

In a real knowledge graph setting, this matters because the quality of the final triplet depends on all three parts being correct.

The EDA helped explain some of the model behavior. ADKG was sparser, with many sentences containing no annotated relations. This likely made the model see many negative examples. MDKG was denser and had more entities and relations per sentence. MDKG also had longer relation distances on average. Because of this, the MDKG model had to consider more possible entity pairs and connect entities across longer spans of text. Even though MDKG had stronger NER performance than ADKG, its final relation extraction performance was slightly lower when entity classification was required.

The SpERT baseline was the stronger structured extraction method overall. It gave stable performance on both datasets and handled clear relation patterns, especially abbreviation relations, fairly well. However, it still struggled with harder biomedical relations such as `risk_factor_of`, `treatment_for`, and `associated_with`. These relation types often require more sentence-level understanding than abbreviation relations.

The T5 extension showed that triplet extraction can also be treated as a generative task. The model learned the output format and produced some exact correct triplets. It also correctly predicted `none` for many no-relation examples. However, the generative approach had more issues with hallucination, incomplete predictions, and wrong relation types. The relaxed F1 score was higher than the strict F1 score, which shows that some predictions were close to correct but not exact.

Overall, the discriminative SpERT model was better for controlled structured extraction, while the generative T5 model was useful as an exploratory extension. The generative model made it easier to inspect outputs as text, but it was less reliable when exact triplet structure mattered.

10. Limitations and Future Work

There are several limitations to this project. First, the final SpERT baseline used `bert-small` instead of a larger biomedical language model. A model like BioBERT or PubMedBERT would likely be a better fit for biomedical text. I used `bert-small` because it trained quickly and reliably within the project timeline, especially after the earlier environment and package issues on Longleaf.

Second, the generative extension used T5-small and only a subset of the combined ADKG and MDKG data. This kept the experiment manageable, but it also limited the possible performance of the generative model. Training on the full dataset for more epochs, or using a larger instruction-tuned model, could improve the generative results.

Third, the comparison between SpERT and T5 was not perfectly controlled. SpERT was evaluated separately on the full ADKG and MDKG test sets, while T5 was evaluated on a smaller combined test subset. This means the numbers

should be interpreted as a comparison of modeling styles rather than a perfectly direct head-to-head benchmark.

Fourth, the relaxed matching metric was simple. It required the relation type to match and the subject and object to share at least one word with the gold subject and object. This was useful as a first attempt, but it does not fully handle biomedical synonyms, abbreviations, or slightly different entity boundaries. A better future evaluation could use biomedical dictionaries, synonym matching, or entity normalization.

Finally, the project focused mostly on sentence-level extraction. Some biomedical relations may require context from surrounding sentences or the full abstract. A future version of this project could explore document-level relation extraction, better handling of long-range dependencies, or combining discriminative and generative models.

11. Conclusion

In this project, I built a full biomedical knowledge graph triplet extraction pipeline using the ADKG and MDKG datasets. I started by exploring the data, checking dataset structure, and analyzing entity types, relation types, sentence density, and relation distance. This helped show that ADKG and MDKG had different challenges. ADKG was sparser and had many no-relation sentences, while MDKG was denser and had longer-distance relations.

For the main baseline, I converted the data into SpERT format and trained separate SpERT models for ADKG and MDKG using a BERT-small encoder. The SpERT baseline performed better on named entity recognition than on relation extraction, which showed that connecting entities into correct triplets was the harder part of the task. On the test sets, the final relation F1 scores with named entity classification were 28.86 for ADKG and 24.95 for MDKG.

For the extension, I trained a T5-small generative model to produce triplets directly as text. This model reached a strict triplet F1 of 19.79 and a relaxed triplet F1 of 26.60 on the combined test subset. The generative model was able to produce some correct triplets and correctly output **none** for many no-relation sentences, but it also missed relations and sometimes generated plausible but unannotated triplets.

Overall, this project showed that biomedical triplet extraction is feasible but challenging. The structured SpERT baseline was more reliable for exact extraction, while the generative T5 extension showed a flexible alternative that was easier to inspect but more error-prone. The results suggest that future improvements would likely come from stronger biomedical encoders, better handling of dense multi-relation sentences, and more advanced evaluation methods for generated triplets.

References

- Eberts, M., & Ulges, A. (2019). Span-based Joint Entity and Relation Extraction with Transformer Pre-training.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.